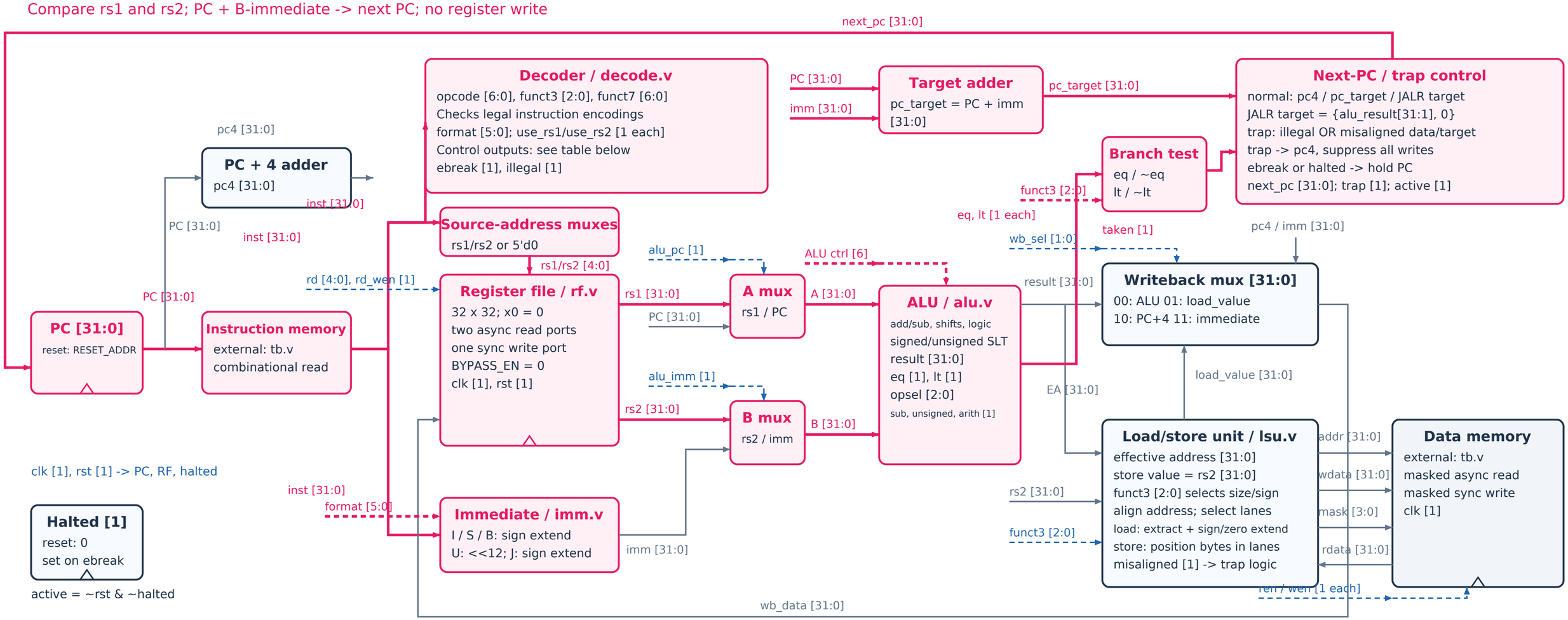
B-type: beq x1, x2, +16 (taken)

Compare rs1 and rs2; PC + B-immediate -> next PC; no register write



Triangles mark clocked state. Gray memory blocks are external to hart. Unmarked blocks are combinational.

Blue dashed wires are control. Matching named nets are electrically connected; bus widths are shown in brackets.

Bright pink highlights the sample instruction path, including writeback and PC update.

Decoder outputs and selection logic

Instruction	format	ALU A / B	ALU op	wb_sel	RegW	Memory / PC
R arithmetic	000001	rs1 / rs2	funct3	00	1	none / pc4
I arithmetic	000010	rs1 / imm	funct3	00	1	none / pc4
Load	000010	rs1 / imm	ADD	01	1	read / pc4
Store	000100	rs1 / imm	ADD	--	0	write / pc4
Branch	001000	rs1 / rs2	compare	--	0	none / target*
LUI	010000	unused	unused	11	1	none / pc4
AUIPC	010000	PC / imm	ADD	00	1	none / pc4
JAL	100000	unused	unused	10	1	none / target
JALR	000010	rs1 / imm	ADD	10	1	none / ALU&~1
EBREAK / FENCE	000001	unused	unused	--	0	hold / pc4

* branch target selected only if the comparison is true; otherwise pc4.
sub: R-type SUB only; arith: SRA/SRAI; unsigned: SLTU/SLTIU or BLTU/BGEU.
use_rs1/rs2 [1 each] select inst[19:15]/inst[24:20], or 5'd0, into RF.
rd_wen = RegW & active & ~trap & (rd != 0); rd = inst[11:7].
ren/wen = load/store & active & ~trap; illegal encodings have no side effects.

Memory lanes, branch conditions, and state control

EA [31:0] = rs1 + immediate; offset [1:0] = EA[1:0].
o_dmem_addr [31:0] = {EA[31:2], 2'b00}.
Byte mask [3:0]: offset 0/1/2/3 -> 0001/0010/0100/1000.
Half mask [3:0]: offset 0/2 -> 0011/1100; word mask = 1111.
Store mux [31:0]: rs2 << 0/8/16/24 selected by offset [1:0].
Load mux [31:0]: rdata >> 0/8/16/24 selected by offset [1:0].
LB/LH sign-extend low 8/16 bits; LBU/LHU zero-extend; LW passes 32 bits.
All lane shifts use constant wiring and muxes, never variable shift operators.
misaligned [1] = half & EA[0] OR word & (EA[1:0] != 0).
BEQ/BNE select eq/~eq; BLT/BLTU select lt; BGE/BGEU select ~lt.
Jump or taken branch traps if target[1:0] != 0 (after JALR clears bit 0).
Trap policy: suppress register/memory writes; continue at pc4 (no CSRs).
EBREAK: assert halt this cycle; hold PC; latch halted on rising clock.
Halted: no memory access, register writes or retirement until reset.
Synchronous reset: PC <- RESET_ADDR; RF <- 0; halted <- 0.

Named control nets: source -> destination (port connections)

decode.use_rs1/use_rs2 [1 each] -> source-address mux selects.
decode.format [5:0] -> imm.i_format; inst [31:0] -> decode.i_inst / imm.i_inst.
decode.alu_pc / alu_imm [1 each] -> A / B mux selects.
decode.alu_op [2:0], sub/unsigned/arith [1 each] -> ALU ctrl [5:0].
decode.wb_sel [1:0] -> writeback mux; decode.reg_write [1] -> rd_wen gate.
decode.load/store [1 each] -> ren/wen gates and data-alignment trap gate.
decode.branch/jal/jalr [1 each] -> redirect / target mux in next-PC logic.
decode.ebreak [1] -> halted set input and next-PC hold select.
decode.illegal [1] -> trap OR; trap [1] -> write/access gates and next-PC mux.

Named data nets and next-PC mux inputs

PC [31:0] -> instruction address, PC+4 adder, target adder, ALU A mux.
imm [31:0] -> ALU B mux, target adder, writeback input 11.
pc4 [31:0] -> next-PC default and writeback input 10.
alu_result [31:0] -> JALR target clear-bit-0 wiring, LSU, writeback input 00.
rs2_data [31:0] -> LSU store-data input and ALU B mux.
LSU.misaligned [1] / target[1:0] / PC[1:0] -> trap detection.
redirect [1] = jal | jalr | (branch & branch_condition).
next_pc = (halted | ebreak) ? PC : trap ? pc4 : redirect ? target : pc4.
halted_next [1] = halted | ebreak; clk/rst [1 each] -> all state elements.